Système d'exploitation : TP1+ (à faire si vous avez terminé le TP1)

février 2026 — Pierre Rousselin

Mots-clés :

— structures

— pointeurs

— fichiers, répertoires et inodes sous Unix

— autonomie et manuel

— récursivité

Exercice 1 : `mon_stat`

L'appel système `stat` permet de consulter l'inode d'un fichier et donc de récupérer des informations très importantes comme le type du fichier ou les permissions.

Le but du jeu est d'écrire un programme `mon_stat` qui affiche des informations sur les fichiers donnés en argument.

1. Dans une première version, afficher, pour chaque fichier donné en argument, le nom, la taille en octets et le nombre de liens.

Voir `man 2 stat`. Le second argument est l'adresse d'une variable de type `struct stat` qui sera remplie si l'appel n'échoue pas.

Dans le cas où l'appel échoue (voir la page de manuel, partie « valeur de retour »), indiquer avec `perror` un message sur le terminal.

2. Les informations cruciales que sont le type de fichier (répertoire, fichier normal, lien symbolique, etc) et les permissions sont contenues dans le champs `st_mode` de la `struct stat`. Problème : ça fait deux informations codées dans un seul entier. Voir la page `man 7 inode`, puis améliorer votre programme de manière à afficher le type du fichier (`-` pour fichier normal, `d` pour répertoire, ...). Pour le type du fichier vous pouvez utiliser les macros `S_ISREG`, `S_ISDIR`, etc.

3. (Bonus) Afficher aussi ses permissions (par exemple en octal). Pour les permissions, il faut faire un masque « et bit à bit » de la façon suivante : si `m` est une variable de type `mode_t`, les permissions sont données par `m & 0777`, où `0777` est l'entier, écrit en octal, ayant ses 9 derniers bits à 1 (et les autres à 0). Plus d'infos sur les opérations bit à bit ici :
`https://fr.wikipedia.org/wiki/Op%C3%A9ration_bit_%C3%A0_bit`
et sur la notation octale des permissions ici :
`https://fr.wikipedia.org/wiki/Permissions_UNIX#Repr%C3%A9sentation_des_droits`.

--- * ---

Exercice 2 : `mon_find`

Maintenant qu'on sait consulter l'inode d'un fichier, on peut savoir si c'est un répertoire et... on peut chercher récursivement un fichier dans une sous-arborescence !

1. À l'aide de `stat`, écrire une fonction

```
int is_dir(const char *parent_dir, const char *basename);
```

qui retourne 1 si le fichier `basename` contenu dans `parent_dir` est de type répertoire, 0 sinon (-1 en cas d'échec de `stat` avec message d'erreur approprié).

Vous aurez à concaténer les deux arguments, séparés par un `/` (utiliser par exemple un grand tableau de caractères dans la pile de `is_dir` ou bien un tableau alloué dynamiquement pour recueillir cette nouvelle chaîne).

Tester cette fonction dans un programme.

2. La commande `find` permet de chercher un fichier satisfaisant certains critères dans une sous-arborescence. Nous allons en écrire une version minimale.

La commande

```
$ mon_find repertoire_de_depart nom_de_base
```

devrait trouver tous les fichiers dont le nom de base est `nom_de_base` dans `repertoire_de_depart`, ou bien dans un de ses sous-répertoires, ou bien dans un sous-sous-répertoire, ... et afficher les chemins de ces fichiers.

Par exemple, une telle sortie pourrait être

```
$ mon_find /home/bob a.out
/home/bob/a.out
/home/bob/prog/a.out
/home/bob/prog/tp1/a.out
/home/bob/truc/a.out
```

Indication : on pourra utiliser une fonction récursive

```
void find(const char *prefixe, const char *dirname, const char *filename);
```

qui cherche dans `prefixe/dirname` le fichier `filename` et s'appelle récursivement sur les sous répertoires de `prefixe/dirname`, en mettant convenablement à jour `prefixe`...

Ce n'est pas si simple ! Ignorer les liens symboliques (et les éventuels messages d'erreur qu'ils provoquent), au moins dans un premier temps.

--- * ---